

1. Runde

des

43. Bundeswettbewerbs Informatik

Aufgabe 2: Schwierigkeiten

Teamname: EZ

Team-ID: 74405

Bearbeitet

von

Florian Bange

Teilnahme-ID: 00259

18. November 2024

Inhaltsverzeichnis

0	Einleitung	2
1	Modellierung des Problems	2
2	Loesungsvorschlag	2
2.1	Aufgabenteil 1: Keine Konflikte	2
2.2	Aufgabenteil 2: Mit Konflikten	2
3	Analyse der Zeit- und Platzkomplexitaet der Loesung	3
4	Implementierung	3
5	Beispiele	4
5.1	Beispiel 1 - „schwierigkeiten0.txt“	4
5.2	Beispiel 2 - „schwierigkeiten1.txt“	4
5.3	Beispiel 3 - „schwierigkeiten2.txt“	4
5.4	Beispiel 4 - „schwierigkeiten3.txt“	4
5.5	Beispiel 5 - „schwierigkeiten4.txt“	4
5.6	Beispiel 6 - „schwierigkeiten5.txt“	4
6	Quellcode	5

0. Einleitung

Dieses Dokument beinhaltet meine Dokumentation der zweiten Aufgabe der ersten Runde des 43. Bundeswettbewerbs Informatik. Fuer diese Aufgabe wird zunaechst das gegebene Problem kurz mathematisch modelliert. Darauf folgt ein Loesungsvorschlag, welcher anschliessend bezueglich seiner Zeit- und Platzkomplexitaet analysiert wird. Zuletzt werden Details zur Implementierung in C++ gegeben, woraufhin die Beispiele der BwInf Website und der Quellcode folgen.

1. Modellierung des Problems

Wie in der Aufgabenstellung seien $n \in \mathbb{N}$ die Anzahl der Klausuren, $m \in \mathbb{N}$ die Anzahl der Aufgaben und $k \in \mathbb{N}$ die Anzahl der Aufgaben, fuer welche eine gute Anordnung gefunden werden soll. Nun sei V die Menge der m verschiedenen Aufgaben. Das Problem laesst sich optimal durch einen gerichteten Graphen modellieren, bei welchem V die Menge der Knoten ist und eine gerichtete Kante von A nach B existiert genau dann, wenn A in einer der n Klausuren direkt vor B steht (d.h., dass mindestens eine der n Zeilen die Form $\dots A < B \dots$ hat). Dadurch kann man eine Aufgabe A als einfacher als B bezeichnen, wenn ein Weg von A nach B im Graphen existiert. Ein Konflikt zwischen zwei Aufgaben, bzw. Knoten A und B besteht nun genau dann, wenn es im Graphen einen Weg von A nach B und einen Weg von B nach A gibt. In diesem Fall existiert also mindestens ein Zyklus im Graphen.

2. Loesungsvorschlag

2.1. Aufgabenteil 1: Keine Konflikte

In diesem Fall handelt es sich bei dem Graphen um einen gerichteten, azyklischen Graphen (im Folgenden DAG). Ein solcher laesst sich stets topologisch sortieren. Das heisst es existiert eine Anordnung $(v_{\pi(1)}, \dots, v_{\pi(n)})$ aller Knoten des Graphen, sodass niemals ein Weg von $v_{\pi(j)}$ nach $v_{\pi(i)}$ existiert, wenn $i < j$. Anders ausgedrueckt sind die Aufgaben durch eine topologische Sortierung so angeordnet, dass fuer zwei Aufgaben A und B , wobei A vor B in der Anordnung steht, niemals B einfacher ist als A . Dabei ist es insbesondere moeglich, dass die Aufgabe unabhaeig von einander sind. Hat man eine solche topologische Sortierung berechnet, so laesst sich die Anordnung fuer die k gewuenschten Aufgaben sofort ablesen, indem alle anderen Aufgaben der Anordnung ignoriert werden.

Eine topologische Sortierung laesst sich einfach mittels Kahns Algorithmus berechnen¹. Dieser Algorithmus geht wie folgt vor: Fuer jeden Knoten des Graphen wird berechnet, wie viele eingehende Kanten er hat. Daraufhin werden alle Knoten ohne eingehende Kante zu einer Queue hinzugefuegt. Solange diese Queue Knoten enthaelt wird jeweils ein Knoten v der Knoten in der Queue betrachtet, aus der Queue entfernt und zur topologischen Sortierung hinzugefuegt. Anschliessend wird v aus dem Graphen entfernt und jeder der von v durch eine Kante erreichbaren Knoten u betrachtet. Jede dieser Kanten wird aus dem Graphen entfernt, sodass sich die Anzahl an eingehenden Kanten von u um 1 verringert. Falls u nun auch 0 eingehende Kanten hat, wird u zur Queue hinzugefuegt.

Dieses Verfahren findet in einem DAG stets eine topologische Sortierung. Denn per Konstruktion werden stets Knoten zur Sortierung angefuegt von welchen aus keiner der Knoten, die bereits in der Sortierung waren, erreicht werden kann. Ausserdem existiert stets mindestens ein Knoten in der Queue, bis alle Knoten zur Sortierung hinzugefuegt worden sind. Denn wenn noch nicht alle Knoten zur Sortierung hinzugefuegt worden sind, aber die Queue leer ist, dann betrachte man zu diesem Zeitpunkt einen Teilgraphen G' des Anfangsgraphen G , sodass G' ebenfalls keine Zyklen besitzen kann. Haette G' aber keine Zyklen, so liesze sich wie folgt ein Knoten ohne eingehende Kante finden: Angenommen, jeder Knoten in G' haette eine eingehende Kante. Dann beginne bei einem beliebigen Knoten u_1 . Dann existiert eine Kante von einem Knoten u_2 zu u_1 . Fuer einen Knoten u_i dieser Folge existiert nun stets ein Knoten u_{i+1} mit Kante von u_{i+1} nach u_i . Waere u_{i+1} einer der Knoten $u_1, \dots, u_i$, so wuerde ein Zyklus existieren. Also ist u_{i+1} ein neuer Knoten. Dieses Verfahren laesst sich beliebig lange fortfuehren. Aber G' hat nur endlich viele Knoten. Widerspruch!

2.2. Aufgabenteil 2: Mit Konflikten

In diesem Fall enthaelt der Graph mindestens einen Zyklus. Wir wollen nun zwei Knoten bzw. Aufgaben als gleichschwer, bzw. aquivalent interpretieren, wenn A und B in einem gemeinsamen Zyklus sind. Dadurch ent-

¹Siehe auch https://en.wikipedia.org/wiki/Topological_sorting

steht eine Äquivalenzrelation auf den Knoten. Also auch eine Partition. Diese Partitionen nennen sich starken Zusammenhangskomponenten (im Folgenden SCC). Dabei handelt es sich um eine maximale Teilmenge an Knoten des Graphen, welche jeweils paarweise gegenseitig von einander erreichbar sind². Im Kontext der Aufgabenstellung handelt es sich bei den starken Zusammenhangskomponenten des Graphen also um Aufgaben, die paarweise gleich schwer sind. Nun lässt sich jeder gerichtete Graph in seine starken Zusammenhangskomponenten zerlegen. Dabei lässt sich Aufgabenteil 1 als Sonderfall auffassen, bei welchen alle SCCs genau einen Knoten enthalten. Nun kann beispielsweise Tarjans Algorithmus zur Bestimmung starker Zusammenhangskomponenten³ benutzt werden. Hat man den Graphen in SCCs zerlegt, so kann man aus diesen SCCs wieder einen Graphen bilden, wobei eine gerichtete Kante von einer SCC C_1 zu einer weiteren C_2 genau dann existiert, wenn ein Knoten in C_1 eine gerichtete Kante zu einem Knoten in C_2 hat. Dann lässt sich bemerken, dass der daraus resultierende Graph ein DAG ist. Denn sonst gäbe es zwei SCCs, die jeweils von einander erreichbar sind, sodass es ebenfalls einen Knoten u in C_1 und einen Knoten v in C_2 gibt, sodass u und v im ursprünglichen Graphen voneinander erreichbar sind. Dann wären aber u und v in derselben SCC, also $C_1 = C_2$. Somit lässt sich auf den neuen Graphen der SCCs das Verfahren aus Aufgabenteil 1 anwenden⁴. Das heisst, es lässt sich eine topologische Sortierung der SCCs finden, die der Ordnung der gleichschweren Klausuren nicht widerspricht. Ordnet man jeweils den Aufgaben einer SCC eine beliebige Reihenfolge zu, so entsteht dadurch eine sinnvolle Anordnung der Aufgaben. Wie oben lässt sich diese Anordnung auf die k gewünschten Aufgaben reduzieren, indem alle anderen Aufgaben ignoriert werden.

3. Analyse der Zeit- und Platzkomplexität der Lösung

Im Folgenden soll nur Aufgabenteil 2 analysiert werden, da dadurch impliziert auch Aufgabenteil 1 betrachtet wird. Es sei nun anders als in Kapitel 1 $n \in \mathbb{N}$ die Anzahl der unterschiedlichen Aufgaben und $m \in \mathbb{N}$ die Summe der Aufgaben in den Zeilen, die die Klausuren darstellen und $k \in \mathbb{N}$ mit $k \leq n$ die Anzahl der gewünschten Aufgaben. Dann wird zunächst ein Graph mit n Knoten und m Kanten erstellt, was in Laufzeit $\mathcal{O}(n + m)$ geschehen kann. Anschliessend wird Tarjans Algorithmus genutzt, welcher ebenfalls eine Laufzeit von $\mathcal{O}(n + m)$ hat. Anschliessend wird ein neuer Graph erstellt, welcher $\leq n$ Knoten und $\leq m$ Knoten hat. Dazu müssen insbesondere alle Kanten des ersten Graphen betrachtet werden. Somit ergibt sich hier ebenfalls eine Laufzeit von $\mathcal{O}(n + m)$. Zuletzt wird Kahn's Algorithmus auf den zweiten Graphen angewendet. Dies benötigt ebenfalls lineare Zeit $\mathcal{O}(n + m)$. Insgesamt ergibt sich also eine lineare Laufzeitkomplexität von $\mathcal{O}(n + m)$. Weiter ist die Platzkomplexität ebenfalls $\mathcal{O}(n + m)$, da die beiden Graphen gespeichert werden müssen und sowohl Tarjans Algorithmus als auch Kahns Algorithmus mehrere Datenstrukturen nutzen, die linear in $n + m$ viel Platz beanspruchen.

4. Implementierung

Die vorgestellte Lösung wurde in C++ implementiert. Dabei wurden davon ausgegangen, dass es niemals mehr als 26 Aufgaben geben würde, da diese in der Eingabe als Buchstaben von A bis Z angegeben sind. Diese Darstellung wurde von A bis Z auf 0 bis 25 übertragen. Die Graphen wurden jeweils als Adjazenzliste implementiert, welche konkret ein Vektor des Typs `set<int>` ist. Denn dadurch muss nicht beachtet werden, dass in der Eingabe Kanten mehrfach vorkommen können.

²Siehe auch https://en.wikipedia.org/wiki/Strongly_connected_component

³Siehe auch https://en.wikipedia.org/wiki/Tarjan's_strongly_connected_components_algorithm

⁴Noch einfacher ist es zu benutzen, dass Tarjans Algorithmus die SCCs einer umgekehrten topologischen Sortierung findet.

5. Beispiele

Nun wird auf die Beispiele der BwInf Website eingegangen. Dabei wird stets eine moegliche topologische Sortierung des Graphs der starken Zusammenhangskomponenten, sowie eine daraus resultierende Reihenfolge der Klausuren angegeben. Eine Komponente wird dabei durch ihre Klausuren beschrieben.

5.1. Beispiel 1 - „schwierigkeiten0.txt“

Eine moegliche topologische Sortierung des Graphs der starken Zusammenhangskomponenten sieht wie folgt aus:

$$B \longrightarrow A \longrightarrow E \longrightarrow D \longrightarrow F, G \longrightarrow C$$

Fuer die angegebenen Klausuren, bietet sich somit die Reihenfolge B, E, D, F, C an, wobei alle 5 Klausuren in unterschiedlichen Komponenten liegen.

5.2. Beispiel 2 - „schwierigkeiten1.txt“

Eine moegliche topologische Sortierung des Graphs der starken Zusammenhangskomponenten sieht wie folgt aus:

$$A \longrightarrow B \longrightarrow E \longrightarrow C \longrightarrow G \longrightarrow F \longrightarrow D$$

Fuer die angegebenen Klausuren, bietet sich somit die Reihenfolge A, C, G, F, D an, wobei alle 5 Klausuren in unterschiedlichen Komponenten liegen.

5.3. Beispiel 3 - „schwierigkeiten2.txt“

Eine moegliche topologische Sortierung des Graphs der starken Zusammenhangskomponenten sieht wie folgt aus:

$$A, B, C, D, E \longrightarrow F, G, H$$

Fuer die angegebenen Klausuren, bietet sich somit die Reihenfolge A, B, D, E, F, G an, wobei A, B, D, E sowie F, G jeweils in derselben Komponenten liegen.

5.4. Beispiel 4 - „schwierigkeiten3.txt“

Eine moegliche topologische Sortierung des Graphs der starken Zusammenhangskomponenten sieht wie folgt aus:

$$A, B, C, D \longrightarrow H, I, L \longrightarrow M, N \longrightarrow E \longrightarrow J \longrightarrow F \longrightarrow K \longrightarrow G$$

Fuer die angegebenen Klausuren, bietet sich somit die Reihenfolge A, B, C, D, H, I, L, M, N, E, J, F, K, G an, A, B, C, D, sowie H, I, L, und M, N, jeweils in derselben Komponente liegen.

5.5. Beispiel 5 - „schwierigkeiten4.txt“

Eine moegliche topologische Sortierung des Graphs der starken Zusammenhangskomponenten sieht wie folgt aus:

$$A \longrightarrow B \longrightarrow C \longrightarrow D, E \longrightarrow G, H, I, J, S \longrightarrow F, P, Q, R \longrightarrow K, M, N, O \longrightarrow L \longrightarrow T, V, W, X, Y, Z \longrightarrow U$$

Fuer die angegebenen Klausuren, bietet sich somit die Reihenfolge B, I, F, N, W an, wobei alle 5 Klausuren in unterschiedlichen Komponenten liegen.

5.6. Beispiel 6 - „schwierigkeiten5.txt“

Eine moegliche topologische Sortierung des Graphs der starken Zusammenhangskomponenten sieht wie folgt aus:

$$\begin{aligned} H \longrightarrow R \longrightarrow Z \longrightarrow C, S \longrightarrow Q \longrightarrow E \longrightarrow K \longrightarrow N \longrightarrow M, O \longrightarrow J \longrightarrow L \\ \longrightarrow F \longrightarrow V \longrightarrow A, D, G, P, T, U, X \longrightarrow B \longrightarrow W \longrightarrow I \longrightarrow Y \end{aligned}$$

Fuer die angegebenen Klausuren, bietet sich somit die Reihenfolge H, R, Z, C, S, Q, E, K, N, M, O, J, L, F, V, A, D, G, P, T, U, X, B, W, I, Y an.

6. Quellcode

Nun folgt der Quellcode des wichtigsten Teils Loesung in C++.

Der folgende Teil beinhaltet die Implementierung von Kahn's Algorithmus zum berechnen einer topologischen Sortierung eines Graphens.

```

1  // Topological Sort using Kahn's Algorithm
vector<int> toposort(vector<set<int>> &graph) {
3      int n = graph.size();
      vector<int> in(n, 0);
5      vector<int> nodes; // Nodes sorted in topological order
      queue<int> zero_nodes; // All nodes currently having indegree 0
7
      // Initialize every node's indegree
9      for (int i = 0; i < n; i++) {
          for (int next : graph[i]) {
11             in[next]++;
          }
13     }

15     // Find all nodes which has indegree 0
    for (int i = 0; i < n; i++) {
17         if (in[i] == 0) {
             zero_nodes.push(i);
19         }
    }

21     // Iteratively create the topological order by removing the nodes with indegree 0 from
23     // the graph and adding them into nodes. Then update adjacent node's indegree.
    while (zero_nodes.size()) {
25         int node = zero_nodes.front();
            zero_nodes.pop();
27         nodes.push_back(node);

29         for (int next : graph[node]) {
             in[next]--;
31             if (in[next] == 0) {
                 zero_nodes.push(next);
33             }
        }
35     }

37     return nodes;
}

```

Der folgende Teil beinhaltet die Implementierung von Tarjans Algorithmus zur bestimmung der starken Zusammenhangskomponenten eines Graphen. Dabei werden einige globale Variablen verwendet.

```

// Always assuming the amount of exams to be <= 26 because they are given as chars in A-Z
2 const int MAXN = 26;

4 // Global variables used in Targan to find all SCCs
vector<set<int>> adjacency(MAXN);
6 vector<int> num(MAXN, -1);
vector<int> low(MAXN, -1);
8 stack<int> sta;
vector<bool> in(MAXN, 0);
10 vector<vector<int>> components;

12 int counter = 0;

14 // Find all strongly connected components using Tarjan
void tarjan(int root) {
16     num[root] = counter++;
    low[root] = num[root];
18     sta.push(root);
    in[root] = 1;
20
    for (int next : adjacency[root]) {
22         if (num[next] == -1) {
             tarjan(next);
24         }
        if (in[next]) {
26             low[root] = min(low[root], low[next]);

```

```

    }
28 }

30 if (num[root] == low[root]) {
    vector<int> c;
32     int node;
    do {
34         node = sta.top();
        sta.pop();
36         c.push_back(node);
        in[node] = 0;
38     } while (node != root);

40     components.push_back(c);
    }
42 }

```

Dieser Teil des Quellcodes beinhaltet den Hauptteil des Programm, der die vorherigen Algorithmen nutzt.

```

// Get all SCCs of the graph represented by adjacency
2   for (int i = 0; i < m; i++) {
    if (num[i] == -1) {
4       tarjan(i);
    }
6   }

8   // Map nodes (exams) to their components by index the component's index
   vector<int> components_map(m, -1);
10
   for (int i = 0; i < components.size(); i++) {
12       for (int v : components[i]) {
           components_map[v] = i;
14       }
   }
16
   // Construct new graph using SCCs
18   // -> The components form the nodes (using their index)
   // -> component i has a directed edge to component j if any node in component i has an
20 // directed edge to any node in component j
   vector<set<int>> graph2(components.size());
22
   for (int i = 0; i < components.size(); i++) {
24       // Go through all nodes of current componets
       for (int node : components[i]) {
26           // Go through all the nodes adjacent to node
           for (int next : adjacency[node]) {
28               // Add an edge from component i to next's component (if its not the same component)
               int component_next = components_map[next];
30               if (component_next != i) {
                   graph2[i].insert(component_next);
32               }
           }
       }
34   }
   }

36 // Create a topological sort of the new graph
38 vector<int> order = toposort(graph2);

```